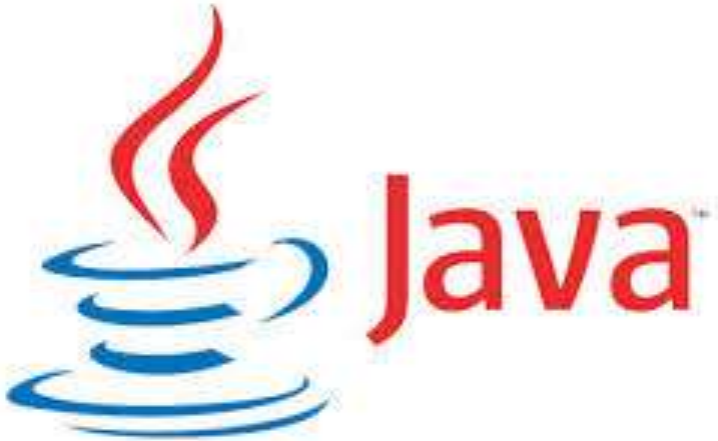


SOC2030 Application Programming in Java

Week 3 (4)

Dr. Andrei Dragunov

Introduction to Classes, Objects, Methods and Strings

- Inheritance
- Specialization
- Associations and Aggregation
- Abstraction
- Polymorphism

Inheritance. Superclasses and Subclasses.

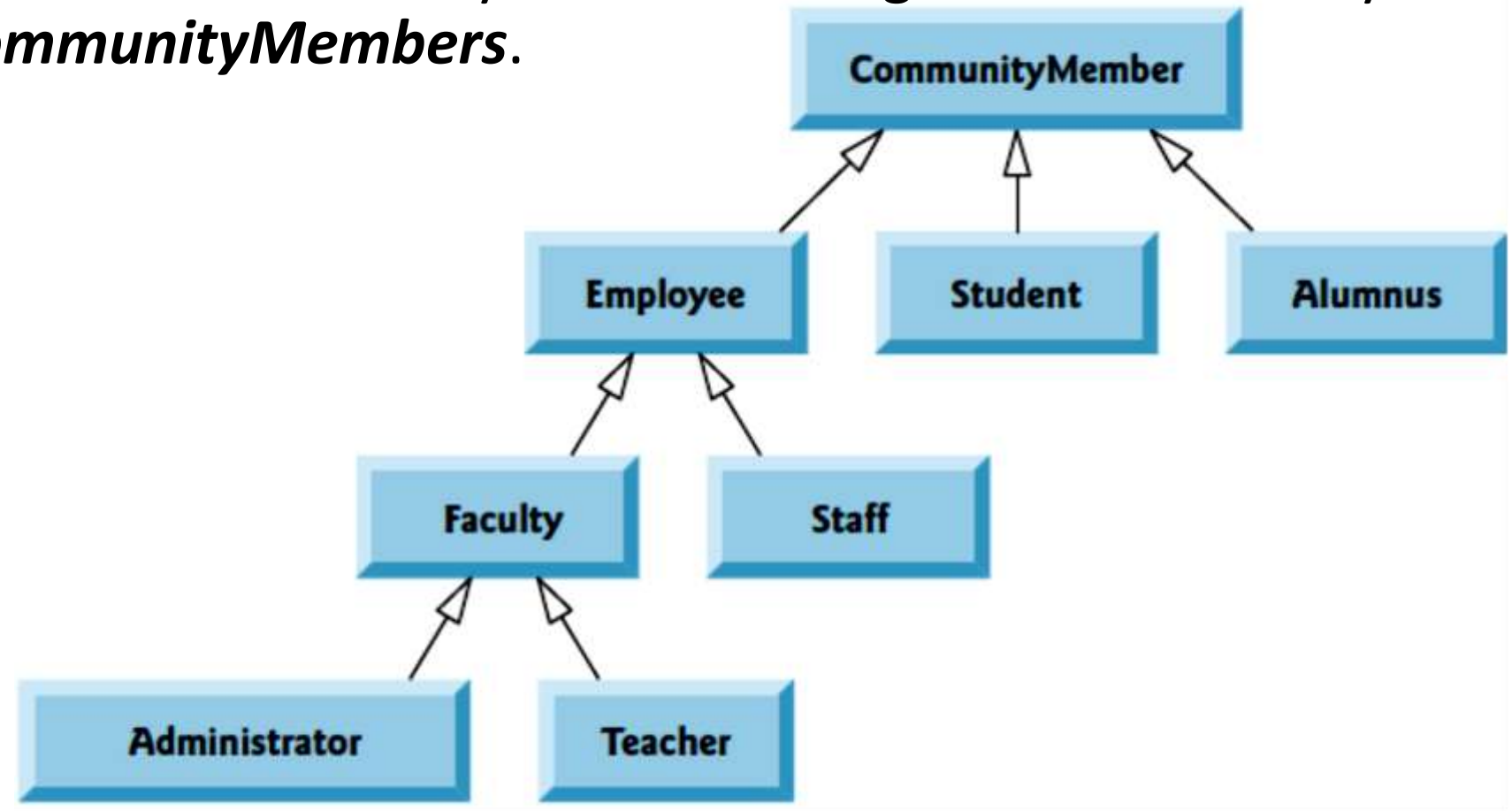
- You use a class to model objects of the same type.

Different classes may have some common properties and behaviors, which can be generalized in a class that can be shared by other classes.

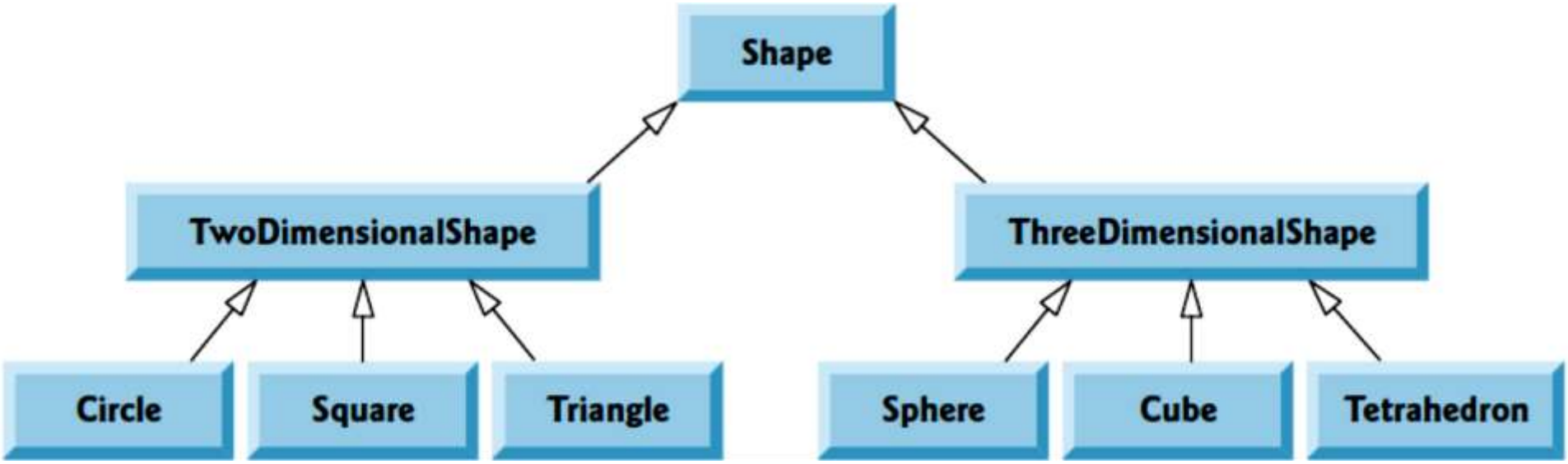
You can define a specialized class that extends the generalized class. The **specialized** classes inherit the properties and methods from the general class.

Superclasses and Subclasses

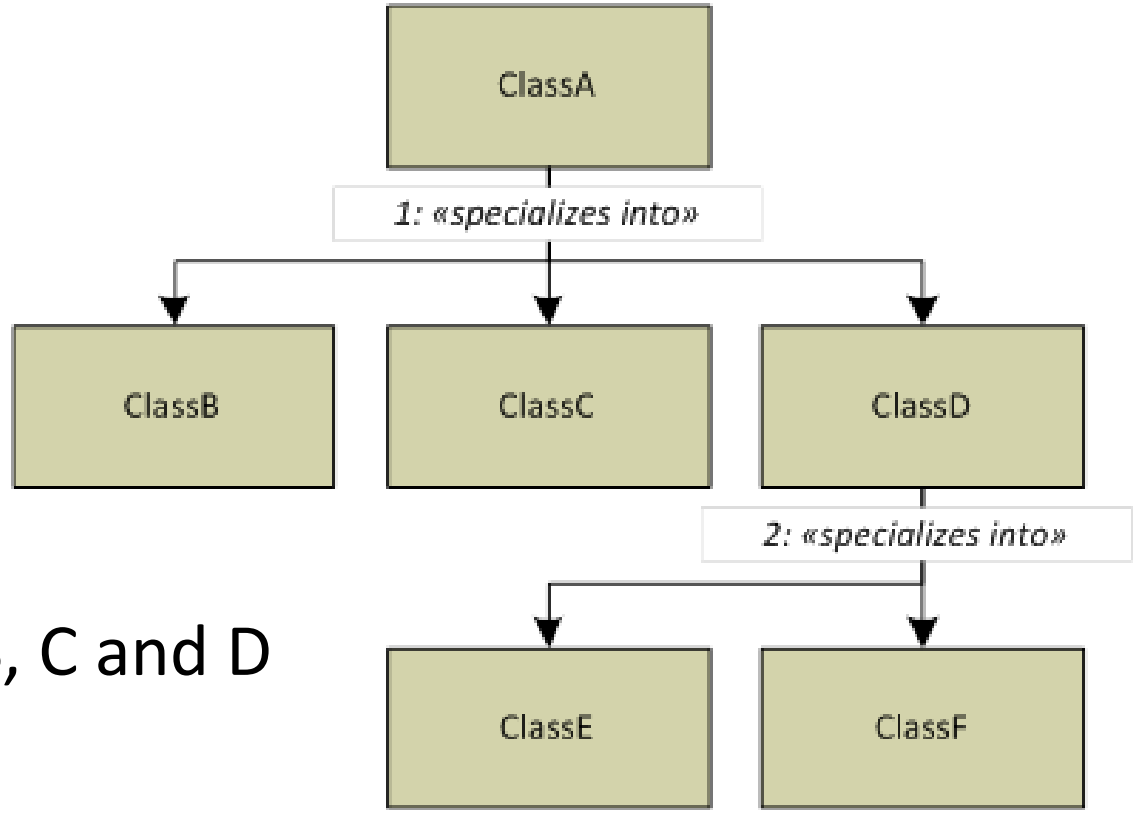
Inheritance hierarchy UML class diagram for university ***CommunityMembers***.



- ***Shape Hierarchy***



Specialization

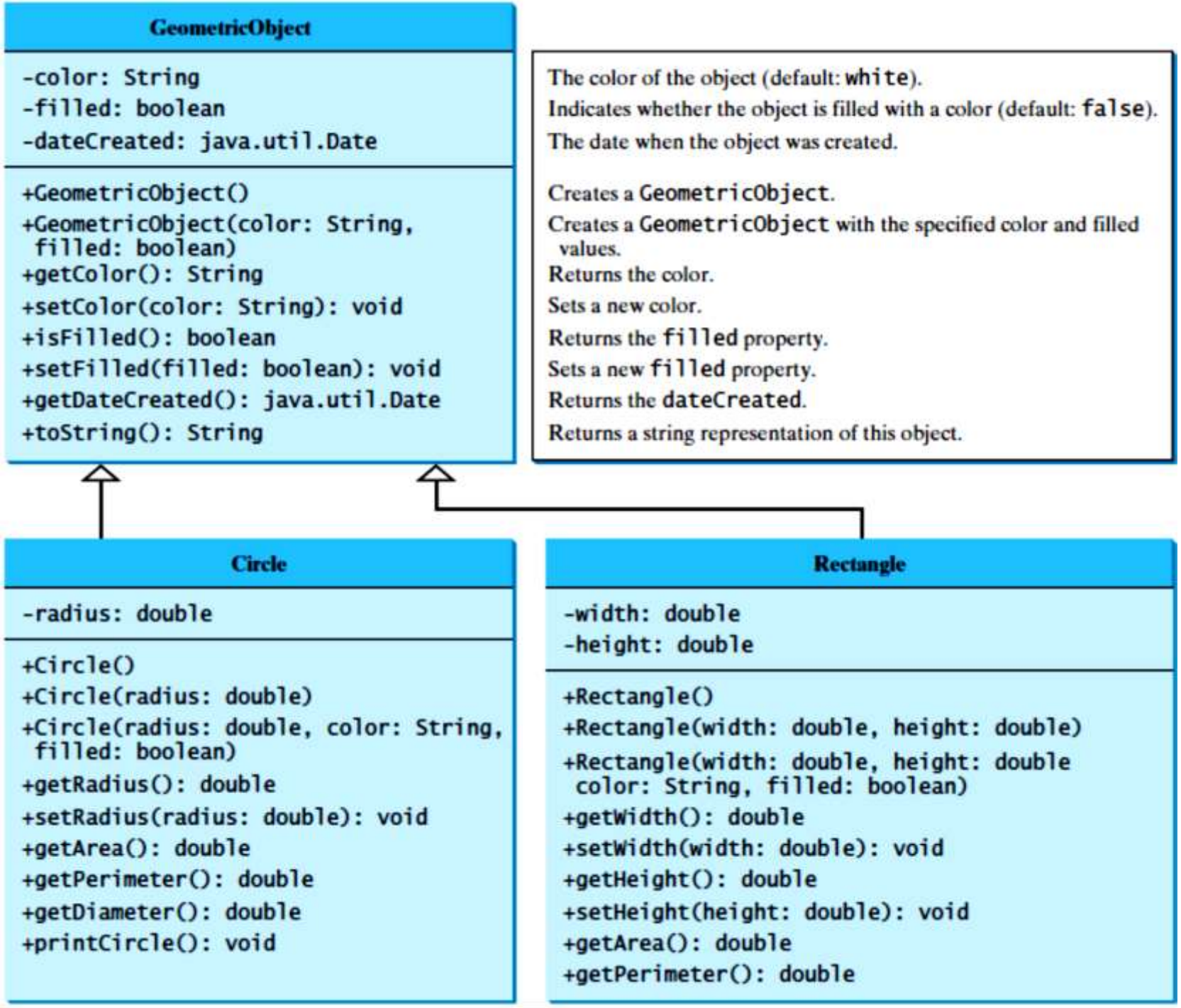


- **Class A**

1: Specializes into Class B, C and D

- **Class D**

2: Specialized further into Class E and F


GeometricObject
-color: String
-filled: boolean
-dateCreated: java.util.Date
+GeometricObject()
+GeometricObject(color: String, filled: boolean)
+getColor(): String
+setColor(color: String): void
+isFilled(): boolean
+setFilled(filled: boolean): void
+getDateCreated(): java.util.Date
+toString(): String
The color of the object (default: white).
Indicates whether the object is filled with a color (default: false).
The date when the object was created.
Creates a GeometricObject.
Creates a GeometricObject with the specified color and filled values.
Returns the color.
Sets a new color.
Returns the filled property.
Sets a new filled property.
Returns the dateCreated.
Returns a string representation of this object.
Circle
-radius: double
+Circle()
+Circle(radius: double)
+Circle(radius: double, color: String, filled: boolean)
+getRadius(): double
+setRadius(radius: double): void
+getArea(): double
+getPerimeter(): double
+getDiameter(): double
+printCircle(): void
Rectangle
-width: double
-height: double
+Rectangle()
+Rectangle(width: double, height: double)
+Rectangle(width: double, height: double color: String, filled: boolean)
+getWidth(): double
+setWidth(width: double): void
+getHeight(): double
+setHeight(height: double): void
+getArea(): double
+getPerimeter(): double

Extends keyword

- The keyword **extends** tells the compiler that the **Circle** class extends the **GeometricObject** class, thus inheriting the methods **getColor**, **setColor**, **isFilled**, **setFilled**, and **toString**.

Note the following points regarding inheritance:

- Contrary to the conventional interpretation, a subclass is not a subset of its superclass. In fact, a subclass usually contains more information and methods than its superclass.
- Private data fields in a superclass are not accessible outside the class. Therefore, they cannot be used directly in a subclass. They can, however, be accessed/mutated through public accessors/mutators if defined in the superclass.

■ Not all is-a relationships should be modeled using inheritance. For example, a square is a rectangle, but you should not extend a **Square** class from a **Rectangle** class, because the **width** and **height** properties are not appropriate for a square. Instead, you should define a **Square** class to extend the **GeometricObject** class and define the **side** property for the side of a square.

■ Inheritance is used to model the is-a relationship. Do not blindly extend a class just for the sake of reusing methods. For example, it makes no sense for a **Tree** class to extend a **Person** class, even though they share common properties such as height and weight. A subclass and its superclass must have the is-a relationship.

■ Some programming languages allow you to derive a subclass from several classes. This capability is known as *multiple inheritance*. Java, however, does not allow multiple inheritance. A Java class may inherit directly from only one superclass. This restriction is known as *single inheritance*. If you use the **extends** keyword to define a subclass, it allows only one parent class.

Using the **super** Keyword

The keyword **super** *refers to the superclass and can be used to invoke the superclass's methods and constructors.* A subclass inherits accessible data fields and methods from its superclass.

- Does it inherit constructors? Can the superclass's constructors be invoked from a subclass?

The keyword **super** refers to the superclass of the class in which **super** appears. It can be used in two ways:

- To call a superclass constructor.
- To call a superclass method.

Calling Superclass Constructors

A constructor is used to construct an instance of a class. Unlike properties and methods, the constructors of a superclass are not inherited by a subclass. They can only be invoked from the constructors of the subclasses using the keyword **super**.

The syntax to call a superclass's constructor is:

super(), or **super**(parameters);

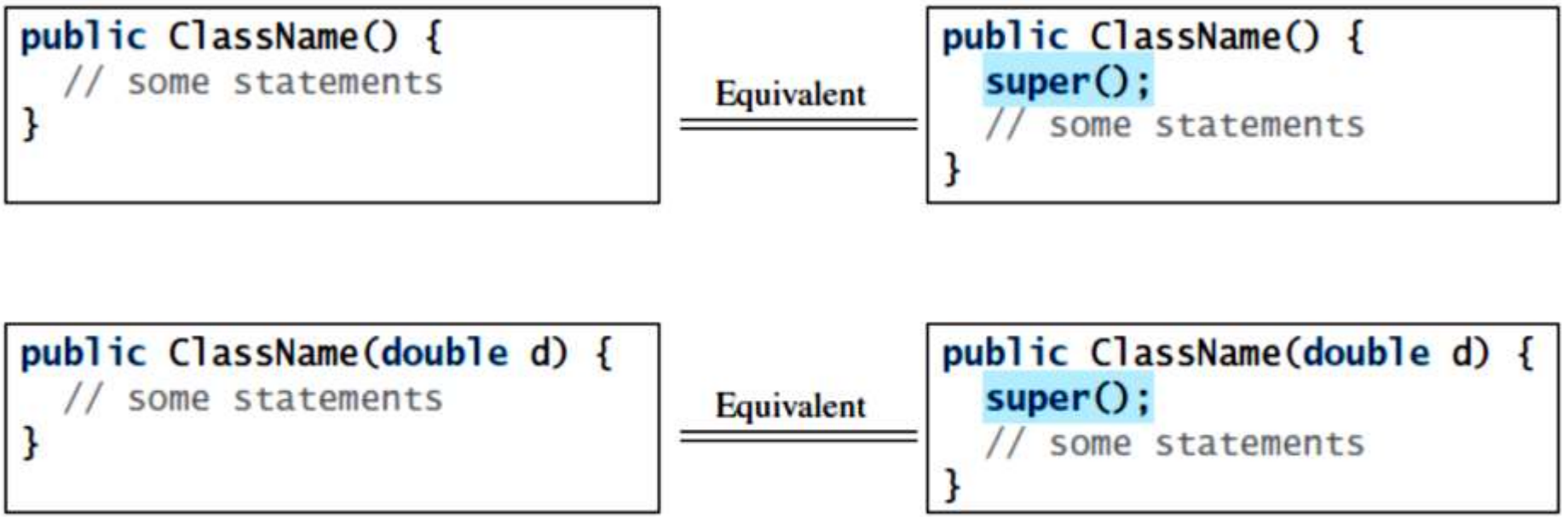
The statement **super()** invokes the no-arg constructor of its superclass, and the statement **super(arguments)** invokes the superclass constructor that matches the **arguments**. The statement **super()** or **super(arguments)** must be the first statement of the subclass's constructor; this is the only way to explicitly invoke a superclass constructor.

Caution

- You must use the keyword **super** to call the superclass constructor, and the call must be the first statement in the constructor. Invoking a superclass constructor's name in a subclass causes a syntax error.

Constructor Chaining

- A constructor may invoke an overloaded constructor or its superclass constructor. If neither is invoked explicitly, the compiler automatically puts **super()** as the first statement in the constructor. For example:



```
class A{
    int a_def;
    public int a_pub;
```

constructor A in class week4_inheritance.A cannot be applied to given types;
required: int
found: no arguments
reason: actual and formal argument lists differ in length

(Alt-Enter shows hints)

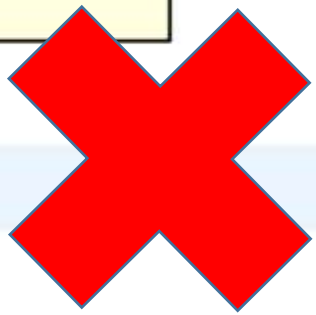
```
    void methodA(int p1,int p2){
        System.out.println("Method A");
    }
}
```

constructor A in class week4_inheritance.A cannot be applied to given types;
required: int
found: no arguments
reason: actual and formal argument lists differ in length

(Alt-Enter shows hints)

```
class B extends A{

}
```



```
class A{
    int a_def;
    public int a_pub;
    private int a_pri;
    protected int a_prot;
    public A(int a){
        a_def = a_pub = a_pri = a_prot = a;
    }
    void methodA(int p1,int p2){
        System.out.println("Method A");
    }
}

class B extends A{
    public B(){
        super(1);
    }
}
```

```
class B extends A{
    public B(){
        super(1);
    }
    void methodB(){
        super.methodA(1, 1);
        System.out.println("Method B");
    }

}
```

ence a method
The syntax is:

```
public static void main(String[] args) {
    A a;
    B b;
    a = new A(1);
    b = new B();
    b.methodB();

}
```

```
class B extends A{
    public B(){
        super(1);
    }
    void methodA(int p1, int p2){
        System.out.println("Method A for class B");
    }
    void methodB(){
        super.methodA(1, 1);
        System.out.println("Method B");
        methodA(1, 1);
```

```
public static void main(String[] args) {
    A a;
    B b;
    a = new A(1);
    b = new B();
    b.methodB();
```

```
run:
Method A
Method B
Method A for class B
BUILD SUCCESSFUL (total time: 0 seconds)
```

- What is the output of running the class **C** in (a)? What problem arises in compiling the program in (b)?

```
class A {
  public A(int x) {
  }
}

class B extends A {
  public B() {
  }
}

public class C {
  public static void main(String[] args) {
    B b = new B();
  }
}
```

(b)

Overriding Methods

- *To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass.*

- An instance method can be overridden **only if it is accessible.** Thus a private method cannot be overridden, because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.
- Like an instance method, a static method can be inherited. However, a static method cannot be overridden. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden. The hidden static methods can be invoked using the syntax **SuperClassName.staticMethodName**.

Overriding vs. Overloading

- *Overloading means to define multiple methods with the same name but different signatures.*
- *Overriding means to provide a new implementation for a method in the subclass.*

```java
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

(a)

```java
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

(b)

Note the following:

- Overridden methods are in different classes related by inheritance; overloaded methods can be either in the same class or different classes related by inheritance.
- Overridden methods have the same signature and return type; overloaded methods have the same name but a different parameter list.

The **Object** Class and Its **toString()** Method

Every class in Java is descended from the **java.lang.Object** *class.*

If no inheritance is specified when a class is defined, the superclass of the class is **Object** by default. For example, the following two class definitions are the same:

```
public class ClassName {
  ...
}
```

Equivalent

```
public class ClassName extends Object {
  ...
}
```

Signature of the **toString()**

```
public static void main(String[] args) {
    A a;
    B b;
    a = new A(1);
    b = new B();
    System.out.println(a.toString());
    System.out.println(b.toString());
}
```

is:

s a string that
ns a string consisting
n instance, an at sign

```
run:
week4_inheritance.A@2a139a55
week4_inheritance.B@15db9742
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
public static void main(String[] args) {
    A a;
    B b;
    a = new A(1);
    b = new B();
    System.out.println(a);
    System.out.println(b.toString());

}
```

Aggregation in Java

- If a class have an entity reference, it is known as Aggregation. Aggregation represents HAS-A relationship.
- Consider a situation, Employee object contains many informations such as id, name, emailId etc. It contains one more object named address, which contains its own informations such as city, state, country, zipcode etc. as given below.

```
class Employee{

int id;

String name;

Address address;//Address is a class

...

}
```

Simple Example of Aggregation

```
class Operation{
 int square(int n){
  return n*n;
 }
}

class Circle{
 Operation op;//aggregation
 double pi=3.14;

 double area(int radius){
   op=new Operation();
   int rsquare=op.square(radius);//code reusability (i.e. delegates the method call).
   return pi*rsquare;
 }
```

When use Aggregation?

- Code reuse is also best achieved by aggregation when there is no is-a relationship.
- Inheritance should be used only if the relationship is-a is maintained throughout the lifetime of the objects involved; otherwise, aggregation is the best choice.

Polymorphism in Java

- **Polymorphism in java** is a concept by which we can perform a *single action by different ways*. Polymorphism means many forms.

There are two types of polymorphism in java:

- compile time polymorphism.
- runtime polymorphism.

We can perform polymorphism in java by method overloading and method overriding.

If you overload static method in java, it is the example of compile time polymorphism. Here, we will focus on runtime polymorphism in java.

Runtime Polymorphism in Java

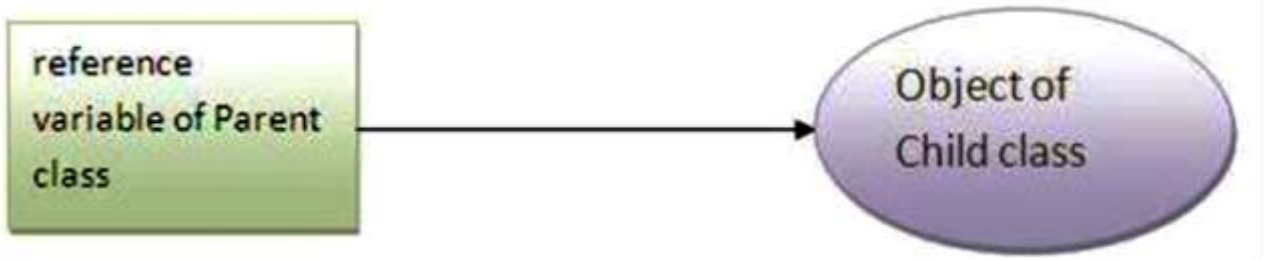
Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Let's first understand the upcasting before Runtime Polymorphism.

Upcasting

When reference variable of Parent class refers to the object of Child class, it is known as upcasting. For example:




```
class A{}

class B extends A{}
```

```
A a=new B();//upcasting
```

The inheritance relationship enables a subclass to inherit features from its superclass with additional new features.

A subclass is a specialization of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa.

For example, every circle is a geometric object, but not every geometric object is a circle. Therefore, you can always pass an instance of a subclass to a parameter of its superclass type.

PolymorphismDemo.java

```
1  public class PolymorphismDemo {
2    /** Main method */
3    public static void main(String[] args) {
4      // Display circle and rectangle properties
5      displayObject(new CircleFromSimpleGeometricObject
6                (1, "red", false));
7      displayObject(new RectangleFromSimpleGeometricObject
8                (1, 1, "black", true));
9    }
10
11   /** Display geometric object properties */
12   public static void displayObject(SimpleGeometricObject object) {
13     System.out.println("Created on " + object.getDateCreated() +
14       ". Color is " + object.getColor());
15   }
16 }
```

polymorphic call

polymorphic call

```
Created on Mon Mar 09 19:25:20 EDT 2011. Color is red
Created on Mon Mar 09 19:25:20 EDT 2011. Color is black
```

```
static public void displayObject(A a){
    System.out.println(a.toString());
}

run:
A
B
C
BUILD SUCCESSFUL (total time: 0 seconds)
```

Static Binding and Dynamic Binding

Connecting a method call to the method body is known as binding.

There are two types of binding

- static binding (also known as early binding).
- dynamic binding (also known as late binding).

Static binding

When type of the object is determined at compiled time(by the compiler), it is known as static binding.

If there is any private, final or static method in a class, there is static binding.

```
class Dog{
 private void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Dog d1=new Dog();
  d1.eat();
 }
}
```

Dynamic binding

When type of the object is determined at run-time, it is known as dynamic binding.

```
class Animal{
 void eat(){System.out.println("animal is eating...");}
}

class Dog extends Animal{
 void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Animal a=new Dog();
  a.eat();
 }
}
```

Dynamic Binding

A method can be implemented in several classes along the inheritance chain. The JVM decides which method is invoked at runtime.

```
public static void main(String[] args) {
    A a = new C();
    System.out.println(a);
}
```

```
run:
C
BUILD SUCCESSFUL (total time: 0 seconds)
```

Method Overloading in Java

- If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.
- If we have to perform only one operation, having same name of the methods increases the readability of the program.
- Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the method such as a(int,int) for two parameters, and b(int,int,int) for three parameters then it may be difficult for you as well as other programmers to understand the behavior of the method because its name differs.

Advantage of method overloading

- **Method overloading *increases the readability of the program*.**

Different ways to overload the method

There are two ways to overload the method in java

- **By changing number of arguments**
- **By changing the data type**

- Implement class structure.
- For each class implement

Superclasses and Subclasses

Inheritance hierarchy UML class diagram for university ***CommunityMembers***.